Intermediate Network Traffic Analysis

Introduction

Mastering network traffic analysis in our constantly evolving and intricate network environments is vital. In this module we will look at a set of attacks that could affect our critical network infrastructure. The goal is to discern patterns and trends within these attacks.

Link Layer Attacks

ARP Spoofing & Abnormality Detection

The Address Resolution Protocol (ARP) has been a longstanding utility exploited by attackers to launch MiTM and DoS attacks among others: ARP often forms a focal point when undertaking analysis for this exact purpose. Many ARP based attacks are broadcasted and not specifically targeted against a host.

In a network. hosts need to know the MAC addresses of the other devices to send their data to. The way they do this is:

1. Computer A needs to talk to computer B and needs its physical address. A consults its ARP cache (list of known addresses) to see if it already possesses the address of B.
2. If the address is not in the ARP cache, A broadcasts an ARP request to the entire subnet asking "Who has the IP address: x.x.x.x?"
3. B responds to the message with an ARP reply, saying "Computer A, my IP is x.x.x.x and my MAC address is aa:aa:aa:aa:aa:aa".
4. A updates its ARP cache.

ARP poisoning and spoofing attacks can be difficult to attack as they mimic the communication structure of standard ARP traffic. Consider a network with three machines: the victim's computer, the router, and the attacker's machine.

1. The attacker initiates their ARP cache by dispatching counterfeit ARP messages.
2. The message to the victim's computer asserts that the gateway's IP address corresponds to the physical address of the attacker's machine.
3. The message to the router claims that the victim's IP address maps to the physical address of the attacker's machine.

4. The attacker aims to corrupt the ARP cache of both the victim's machine and the router to cause all data to misdirect to the attacker's machine. The attacker can then escalate to a MiTM attack by enabling traffic forwarding.
5. The attacker can further escalate the attack by attempting to intercept sensitive traffic data.

To counter this attack we can attempt to disallow easy rewrites of the ARP cache by using static ARP entries. We can also implement network profile controls but both of these measures require increased maintenance and oversight in our network. We can search for attacks by using the Wireshark filter `arp.opcode`, a red flag to watch out or is suspicious requests coming from only one host. We can also filter with `opcode == 1` to view all ARP requests and `opcode == 2` to view all ARP replies. On a Linux system we can search through the ARP cache using grep and the `arp` command:

```
arp -a | grep <MAC-ADDRESS>
```

We can also look for duplicate address mappings in Wireshark by using the filter in Wireshark:

```
arp.duplicate-address-detected && arp.opcode == 2
```

A crucial question we need to ask now is which IP address is legitimate and which address is the one spoofing the ARP cache. Note that one way we can do this is by looking at the historical IP addresses of certain hosts, since it is likely that the victim's address has not changed but that the attacker's address changed recently to the spoofing attack. We can look to filter for these address changes in wireshark:

```
(arp.opcode) && ((eth.src == <MAC>) || (eth.dst == <MAC>))
```

Similarly we could use `eth.addr`. If TCP connections keep dropping then it is an indication that an attacker is not forwarding traffic. However, if we observe identical or symmetrical transmissions from the victim to the attacker and from the attacker to the router then this is a sign of a MiTM attack.

ARP Scanning & Denial-of-Service

Adversaries can also exploit ARP for information gathering. We can try and identify ARP scanning by looking for:

1. Sequential ARP requests.
2. ARP requests sent to non-existent hosts
3. An unusual amount of ARP traffic coming from one host.

An attacker can exploit ARP scanning to compile a list of live hosts which they can then use to deny service to those machines. We might notice the compromised host declaring new addresses for all live addresses to corrupt the router's ARP cache. We may also notice the duplication of the gateway address on client devices which would indicate the attacker attempting to corrupt the ARP cache of the victim devices. Link-layer attacks often fly under the radar so it is important to properly identify and investigate suspicious ARP traffic.

802.11 DoS

In NTA it is important to scrutinise all aspects of link-layer protocols and communications and one very important one is 802.11 (WiFi). It is essential that we continuously audit our wireless networks. To examine WiFi traffic we require a WIDS/WIPS system or wireless interface equipped with monitor mode to view raw 802.11 frames. We can enumerate our wireless interfaces in Linux with the following command:

```
iwconfig
```

We have a few different options for setting our interface into monitor mode:

```
sudo airmon-ng start <interface>
```

```
sudo ifconfig <interface> down
sudo iwconfig <interface> mode monitor
sudo ifconfig <interface> up
```

The crucial factor to look for is `iwconfig` reporting the mode as `Monitor`. It is possible the interface will be renamed. To start capturing traffic from clients and network we can use airodump-ng. We need to specify our AP's channel, its BSSID and our output file name:

```
sudo airodump-ng -c <channel> -bssid <channel> -w <file>
```

We can also use tcpdump to achieve similar outcomes. Among the more frequent attacks we might witness or detect a deauthentication / dissociation attack. This is a common precursor attack that adversaries might employ for several reasons. They could be attempting to capture a password hash for cracking offline, causing general DoS conditions or try to capture credentials through a fake reconnect. In essence, the deauth attack works by spoofing a deauthentication frame from the legitimate access point. The attacker modifies the MAC address of the frame's sender which works since the client device cannot discern the difference without additional controls. Each deauth request is associated with a reason code explaining why the client is being disconnected - most basic tools use code 7.

```
wlan.bssid == xx:xx:xx:xx:xx:xx
```

We can use the above command to limit our view to traffic from our AP's BSSID, this could help us look for deauthentication frames. We can even filter more precisely for this traffic:

```
(wlan.bssid == <bssid>) and (wlan.fc.type == 00) and (wlan.fc.type_subtype == 12)
```

This filter looks specifically for management frames with deauthentication subtypes. We should look for excessive amounts of frames present here and we can inspect these frames and look for the reason code provided.

```
wlan.fixed.reason_code == 7
```

It is generally trivial for an attacker to change the reason code so we need to be careful not to rely on this as a detection method, however we can use it as a potential indicator. Deauthentication can be a pain to deal with but there are some compensating measures we can take:

- We can enabled IEEE 802.11w (Management Frame Protection).
- We can utilise WPA3-SAE.
- We can modify our WIDS/WIPS detection rules.

Suppose an attacker was attempting to connect to our network. We might notice a large amount of association requests coming from one device.

```
(wlan.bssid == <bssid>) and (wlan.fc.type == 00) and (wlan.fc.type_subtype == 0) or (wlan.fc.type_subtype == 1) or (wlan.fc.type_subtype == 11)
```

Rogue Access Point & Evil-Twin Attacks

Rogue access points and evil-twin attacks are significant concerns in our network. A rogue access point primarily serves as a tool to circumvent perimeter controls in place. An adversary might install an access point to sidestep network controls and segmentation barriers with the intention to provide unauthorised access to restricted sections of a network. Rogue access points are directly connected to a network. Evil-twin attacks often contain access points that are not connected to the network which might run a web server or a application to act as a MiTM. Attackers can use this to harvest wireless or domain passwords among other pieces of information. We can use airodump-ng to detect evil-twin style access points:

```
sudo airodump-ng -c <channel> --essid <essid> <interface> -w <file>
```

To ascertain whether we have anomalies or errors we can filter for beacon frames in Wireshark or tcpdump.

```
(wlan.fc.type == 00) and (wlan.fc.type_subtype == 8)
```

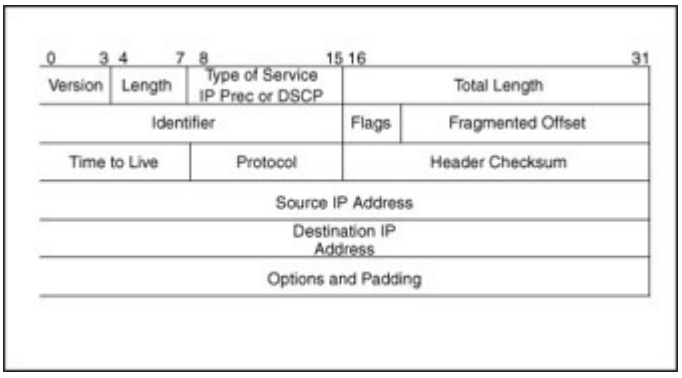
Beacon analysis is crucial in differentiating between genuine and fraudulent access points. A good place to start is the Robust Security Network (RSN) information. This data can communicate valuable information to clients. In most instances, a standard evil-twin attack will not display encryption information in the beacon frame. Nevertheless we should still search for inconsistencies in other fields for more sophisticated evil-twin attacks. Unfortunately despite security awareness training, some users may fall prey to attacks like these. In the case of open network style evil-twin attacks, we can view the highest-level traffic in an unencrypted format.

Detecting Network Abnormalities

Fragmentation Attacks

When we start looking for network abnormalities, we should always consider the IP layer. It is essential to understand that this layer has no mechanisms to identify when packets are lost, dropped or otherwise tampered with. To dissect these packets we can explore some of their fields:

- Length - IP header length
- Total length - IP datagram/packet length
- Fragment offset (provides instructions to reassemble the packet upon delivery)
- Source and destination IP addresses



Traditionally, an attacker might attempt to evade IDS controls through packet malformation/modification. Fragmentation servers as a means for our legitimate hosts to

communicate large amounts of data to one another by splitting the packets and reassembling them. This is commonly achieved using a maximum transmission unit (MTU). The field gives instructions to the destination host on how it can reassemble these packets into a logical order. Attackers may abuse this field in the following way:

- IPS/IDS Evasion: If our detection controls do not reassemble fragmented packets an attacker could split their enumeration into several packets and avoid detection.
- Firewall Evasion: Likewise an attacker could enumerate through a firewall by fragmenting their packets.
- Resource Exhaustion: An attacker could craft their attack to fragment packets to a very small MTU with the intent to either overwhelm the detection system or to prevent it from reassembling the packets due to resource constraints.
- Denial of Service: An attacker could use fragmentation to send packets that are too large to be reassembled which can cause all manner of unexpected behaviour.

To spot irregularities in fragment offsets we can use Wireshark. In particular, multiple ICMP requests from one host to another could indicate the start of an nmap scan.

```
nmap -f 10 <ip>
```

The more notable indicator of a fragmentation scan, regardless of its evasion use, is the single host to many ports issues. In this case, the destination host would respond with RST flags for ports which do not have a service running on them. If Wireshark is not reassembling packets for us, we can change this in the protocol settings.

IP Source & Destination Spoofing Attacks

There are many cases where we might see irregular traffic for IPv4 and IPv6 packets. When analysing source and destination IP fields we should consider if the source IP address is from our subnet. An attacker might conduct packet crafting attacks towards the source and destination IP addresses for many different reasons:

- Decoy scanning: An attacker might change the source IP of packets to bypass firewall restrictions and enumerate a host in another network segment.
- Random source attack DDoS: Through source crafting, an attacker might be able to send tons of traffic to the same port on the victims host to exhaust resources.
- LAND attacks: The source address is set to the same as the destination hosts. In doing so, the attacker might be able to cause crashes. The aim of this attack is to make legitimate connections more difficult to establish by port re-use.

- SMURF attacks: Attackers send large amounts of ICMP packets to many different hosts. In this case, the source address is set to the victim machines such that the constant deluge of ICMP replies causes resource exhaustion on the target.
- Initialisation vector generation: In older wireless networks, an attacker may capture, decrypt, craft, and then re-inject a packet with modified source and destination addresses to generate initialisation vectors to build a decryption table.

Note that these attacks are distinct from ARP poisoning although they tend to be conducted in tandem.

The attacker may attempt to cloak their address with a decoy. However, the responses for multiple closed ports will still be directed towards them with TCP packets with the RST flag set. It can be obvious to notice this when there are large blocks of packets that are all RST. We can also look for connections started by one host and then taken over by another.

To help detect LAND and random source attacks we should:

1. Monitor single port utilisation from random hosts.
2. Monitor incremental base ports with a lack of randomisation.
3. Look for large swathes of packets with identical length fields.

In a SMURF attack, the attacker will send an ICMP request to live hosts with a spoofed address of the victim host. The live hosts will then respond to the legitimate victim host with an ICMP reply. Sometimes attackers will include fragmentation to increase traffic volume even further.

IP TTL Attacks

Time-to-Live attacks are primarily used as a means of evasion by attackers. An adversary will intentionally set a very low TTL on their IP packets to try and get the packet discarded before it reaches a firewall or filtering system. Most times attackers will use TTL manipulation in port scanning. Our best defence against attacks like this is filtering packets based on TTL - it is especially worth noting that common operating systems will use fairly standard TTL values for their packets by default so this can be used as an aid to fingerprint devices.

TCP Handshake Abnormalities

When attackers are gaining information on our TCP services, we might notice some odd behaviour. It is important to have a reference of all TCP flags so that we can better identify strange behaviour:

Flag	Name	Description
URG	Urgent	Denotes urgency with the current data in the stream.
ACK	Acknowledgement	Acknowledges the receipt of data.
PSH	Push	instructs the TCP stack to immediately deliver the received data and bypass buffering.
RST	Reset	Used to terminate the TCP connection.
SYN	Synchronise	Used to establish a connection.
FIN	Finish	Denotes the finish of a connection. No more data needs to be sent.
ECN	Explicit Congestion Notification	Lets hosts know to avoid unnecessary re-transmissions due to congestion in the network.

When analysing our network we can look for unusual flag behaviour such as:

- Too many flags of one type - Can indicate scanning.
- Use of strange flags - Can indicate RST attacks, hijacking or control evasion.
- Solo host to multiple ports/hosts.

In particular we might notice the following behaviour which would alarm us:

- **Excessive SYN flags**: This is a prime indicator of network scanning. The attacker will send TCP SYN packets to the target port which will respond with a SYN-ACK packet if it is open or an RST packet if it is not.
- **No flags**: A scan in which the attacker sends TCP packets with no flags is known as a NULL scan. In a NULL scan, if the target port is open, the system will not respond but if the port is closed the system will respond with an RST packet.
- **Too many ACKs**: The attacker could also employ the use of an ACK scan. If the target port is open it will either not respond or will respond with an RST packet. If the port is closed the system will respond with an RST packet.
- **Excessive FINs**: An attacker might utilise a FIN scan where all packets are marked with the FIN flag, this leads to similar responses to a NULL scan. If the port is open the system will not respond but if it is closed the machine will respond with an RST packet.
- **Too many flags of any type**: The attacker may utilise a Xmas tree scan, where each packet sent has all flags set. Xmas tree scans are very easy to spot since the traffic

generated has no legitimate reason to have all flags set.

TCP Connection Resets & Hijacking

TCP does not provide any protection against our hosts having their connections terminated or hijacked by an attacker. If an attacker wanted to cause denial-of-service conditions within our network they could use a TCP RST packet injection attack:

1. The attacker spoofs the source address of the victim machine's.
2. The attacker modifies the TCP packet to contain the RST flag to terminate the connection.
3. The attacker specifies the destination port to be the same as one currently in use.

We might notice an RST attack through the large amount of packets going to one port and we can verify the attack by looking at the physical address of the transmitter of the TCP RST packets. However bear in mind that attackers can spoof their MAC address so we should also look for re-transmissions and other issues with ARP. More advanced attackers may try to hijack a specific TCP connection. This attack is commonly used alongside ARP poisoning and we can look for lots of re-transmission packets with the PSH flag set.

ICMP Tunnelling

Tunnelling is a technique employed by adversaries in order to exfiltrate data. There are many different kinds of tunnelling and each kind uses a different protocol. Attackers may use proxies to bypass network controls. The idea behind tunnelling is that an attacker will be able to expand their command and control and bypass our network controls through a protocol they chose.

```
ssh -L 9999:localhost:80 user@instance
```

In the case of ICMP tunnelling, an attacker will append data they wish ti exfiltrate in the data field of an ICMP request. The intention is to hide the data inside a common protocol type and get it lost inside the network traffic. We could look for large amounts of ICMP fragmentation happening and view them in closer detail, specifically looking at the data field. We should also view the data field's ASCII representation and look for obvious signs of cleartext transfer or encoded data being sent. To prevent ICMP tunnelling from occurring we can inspect ICMP requests and replies for data or strip it altogether.

Application Layer Attacks

HTTP/HTTPs Service Enumeration

Often we will notice strange traffic to our web servers. Generally speaking we can detect and identify fuzzing attempts by looking for excessive HTTP/HTTPs traffic from one host. Primarily, attackers will attempt to fuzz our server to gather information before launching an attack. We often do not have a web application firewall on internal servers so fuzzing is something we need to watch out for. Directory fuzzing is generally easy to detect, since we expect to see lots of HTTP requests to files or directories that do not exist on our server.

Similarly, we can detect URL fuzzing simply by looking for sequential or otherwise ordered requests. We can also follow the HTTP streams in Wireshark to get a better overview of the attack. Sometimes attackers may stagger their responses or send them from multiple hosts to try and prevent detection.

Strange HTTP Headers

Whilst we may not notice an attacks like fuzzing right away but this does not indicate that nothing is going on. We might want to look for strange HTTP requests which may have unusual headers with:

- Weird hosts
- Unusual HTTP verbs
- Changed user agents

We can filter Wireshark for HTTP requests that have the host header set as something other than the server's real IP address:

```
http.request and (!(http.host == "<legitimate IP>"))
```

Attackers may attempt to use different host headers to gain levels of access they might not achieve normally. They may use proxy tools to modify these requests before sending them to the server. We should ensure our virtualhosts or access configurations are setup to prevent attacker being able to gain greater access by changing the host. We can also look for our server sending HTTP error statuses, we can then follow these streams to see why the server is sending an error. For example the attacker might attempt to perform a CRLF smuggling attack - where the attacker tries to smuggle a second request under the privileges of the first.

```
http.response.code == 400
```

XSS & Code Injection Detection

If we notice lots of requests being sent to an internal server that we do not recognise, this can be an indicator of cross site scripting attacks. In a XSS attack, an attacker will inject malicious

JavaScript or script code into one of our web pages through user input. When other users visit our web server, their browsers will execute this code and we can search for these requests. To prevent XSS attacks we should sanitise and handle user input in accordance to cyber best practices and we should never interpret user input as code.

SSL Renegotiation Attacks

HTTP traffic is not encrypted but we will often encountered HTTPs traffic which is. Because of this, knowing the indicators and signs of malicious HTTPs traffic is crucial. HTTPs incorporates encryption to provide security for web servers and clients, it does so using:

- Transport layer security (TLS)
- Secure sockets layer (SSL)

Generally speaking, when a client establishes a HTTPs connection with a server it conducts the following:

- Handshake (where the client and server agree on encryption algorithms and exchange their certificates)
- Encryption
- Data Exchange
- Decryption (Client and server decrypt using their private keys)

One of the most common HTTPs attacks is SSL renegotiation in which an attacker negotiates the session to the lowest possible encryption standard. In order to filter Wireshark for only handshake messages we can use the following filter:

```
ssl.record.content_type == 22
```

When we are looking for SSL renegotiation attacks we might notice several client hellos from one client within a short period. We also might notice out of order handshake messages. An attacker might conduct this attack against us for Denial of Service purposes, to exploit vulnerabilities with our current implementation of cipher suites or to analyse our SSL/TLS patterns for other systems.

Peculiar DNS Traffic

DNS traffic can be difficult to inspect as often there is lots of it and abnormalities get buried. DNS queries are used when a client wants to resolve a domain nae with an IP address or the other way around. The most common type of DNS query are forward lookup's which work by:

1. The client initiates the query.
2. The client checks its own local DNS cache to see if it has already resolved the domain name to an IP address.
3. The client sends a recursive query to its configured DNS server.
4. The DNS resolver starts by querying the root name servers to find the authoritative name servers from the top level domain.
5. The root server responds with the authoritative name servers.
6. The DNS resolver then queries the authoritative name servers for the second-level domain.
7. The DNS resolver queries the authoritative name servers to obtain the IP address associated with the requested domain.
8. The DNS resolver sends the IP address back to the client.

We also have reverse lookups, where the client knows the IP address and wants to find the FQDN. DNS has many different record types responsible for holding different information:

Record Type	Label	Description
A	Address	Maps a domain name to an IPv4 address
AAAA	IPv6 Address	Maps a domain name to an IPv6 address
CNAME	Canonical Name	Creates an alias for domain name
MX	Mail Exchange	Specifies the mail server responsible for receiving email on behalf of the domain
NS	Name Server	Specifies an authoritative name server
PTR	Pointer	Used in reverse queries to map an IP to a domain name
TXT	Text	Specifies text related to the domain
SOA	Start of Authority	Contains information about the zone

If we see lots of DNS traffic from one host ending with the "ANY" keyword we should be on alert for DNS enumeration or subdomain enumeration. On the other hand, we can look out for DNS tunnelling by looking for lots of DNS text queries. It is uncommon to have large amounts of TXT records and particularly suspicious if these records contain encoded text - this could be a sign

that an attacker is attempting to exfiltrate data through DNS. Sometimes botnets use DNS tunnelling to communicate back to their C2 servers. DNS tunnelling can also be used to bypass firewalls that only monitor HTTP/HTTPs traffic. Over recent years it has been increasingly common that attackers will use the interplanetary file system to store and pull malicious files: we should watch out for DNS/HTTP traffic to IPFS related domain names as these attacks can be difficult to detect since IPFS operates on a peer to peer basis (IPFS).

Strange Telnet & UDP Connections

Telnet is a network protocol that allows a bidirectional interactive communication session between two devices over a network. As of recent years, its usage has decreased significantly due to the popularity of SSH. Older windows machines may still use telnet for remote C2 to Microsoft terminal services. Telnet traffic tends to be decrypted and easily inspectible however attacker can take steps to encode or obfuscate the text. Telnet is just a communication protocol and so can be easily switched to another port, we can keep an eye on strange telnet port communications and follow the TCP streams to glean more information about the conversation.

Attackers may opt to use UDP connections over TCP in their exfiltration efforts. Like TCP, we can follow UDP traffic in Wireshark and inspect it. Attackers might like UDP because it provides quicker connections and is often less scrutinised.